Case Study Report: Turkish Text Classification

Ayça Elif Aktaş

1 Introduction

This case study explores the performance of text classification models for Turkish sentences using both Large Language Models (LLMs) and traditional Machine Learning/Deep Learning (ML/DL) techniques. The dataset used, TWNERTC TC Fine Grained NER DomainIndependent NoiseReduction version, contains 660K sentences and 49 categories. The dataset is imbalanced, presenting an additional challenge. The objective is to assess and compare the accuracy, efficiency, and practicality of these models for deployment. In this report, we focus on the results obtained using LLM, specifically the **Llama 3:8B** model and ML technique **Logical Regression** for comparison.

2 LLM

2.1 Data Exploration and Preprocessing

2.1.1 Exploration

The dataset was first explored to understand the number of categories and the imbalance between them. Each category contained varying numbers of data points, with significant disparities in the distribution across categories. This imbalance was particularly noteworthy in certain domains.

2.1.2 Preprocessing

The following preprocessing steps were applied:

- **Missing Values**: Checked for missing values in the dataset and addressed them accordingly.

- **Cleaning Sentences**: Removed punctuation marks from the sentences to experiment with the LLM model. The goal was to compare performance between the cleaned and original sentence versions. However, due to time and computational constraints caused by hardware issues, this experiment was not completed.
- **Normalization**: Normalized the cleaned sentences for consistency.
- **Duplicate Removal**: Removed duplicate sentences from the dataset to improve the model's accuracy.
- **Data Splitting**: Initially, the dataset was split into 90% training and 10% test data. However, due to computational constraints, a 97.5% training and 2.5% test split was applied instead. The test and training data were saved in TSV format as *cleaned_test_data_ml.tsv* and *cleaned_train_data_ml.tsv*.

2.2 LLM Implementation

2.2.1 Model Choice

For the LLM part of the study, the **Llama 3:8B** model from Ollama was selected, which was run as an API for text classification task. The decision to use this model was informed by the findings from Fechner and Dorpinghaus's study titled *No Train, No Pain? Assessing the Ability of LLMs for Text Classification with no Finetuning*. Their study compared different LLM models on Ollama, including Llama 3:8B, using the zero-shot method without any fine-tuning.

Zero-shot learning refers to the ability of a model to classify or make predictions on tasks it has never been explicitly trained for. In a zero-shot scenario, the model is provided with a task description or context (in this case, a text classification prompt) but does not receive any additional training on the specific dataset. The zero-shot method leverages the general knowledge stored in large language models, enabling them to make reasonable predictions without task-specific optimization.

Based on their results, which showed moderate success with the zero-shot approach (see Figure 1), the Llama 3:8B model was selected for this case study. It struck a balance between performance and computational feasibility, given the constraints of running on local hardware.

Considering these findings, the Llama 3:8B model was chosen for this study, and the results were evaluated based on the direct prompting strategy without any further fine-tuning.

TABLE I
METRICS FOR DIFFERENT LLMS, PROMPTING STRATEGIES AND PROMPT TYPES. LARGE MODELS (LLAMA3:70B AND COMMAND-R:35B) OUTPERFORM SMALLER ONES.

LLM	Strategy	Prompt Type	Metrics (Macro/Weighted)			Failed Samples
			Precision	Recall	F1 Score	
llama3:8b	Direct	zero_shot	0.01 / 0.00	0.12 / 0.01	0.02 / 0.00	0
		one_shot	0.04 / 0.13	0.03 / 0.01	0.01 / 0.02	0
		few_shot	0.11 / 0.61	0.05 / 0.16	0.04 / 0.19	2
	EC	zero_shot	0.21 / 0.73	0.20 / 0.42	0.14 / 0.39	16
		one_shot	0.04 / 0.28	0.07 / 0.08	0.02 / 0.10	0
		few_shot	0.00 / 0.00	0.06 / 0.01	0.00 / 0.00	2
	ECR	zero_shot	0.15 / 0.62	0.22 / 0.54	0.15 / **0.53**	15
		one_shot	0.03 / 0.23	0.08 / 0.16	0.03 / 0.18	0
		few_shot	0.00 / 0.00	0.06 / 0.01	0.00 / 0.00	6
	EHC	zero_shot★	**0.27 / 0.69**	0.21 / 0.52	**0.19** / 0.49	12
		one_shot♦	0.04 / 0.20	0.11 / 0.10	0.103 / 0.11	31
		few_shot♦	0.09 / 0.47	0.14 / 0.04	0.02 / 0.02	84
aya:8b	Direct	zero_shot	0.00 / 0.00	0.03 / 0.01	0.00 / 0.00	0
		one_shot	0.06 / 0.47	0.06 / 0.02	0.00 / 0.00	0
		few_shot	0.03 / 0.25	0.04 / 0.31	0.04 / 0.27	5
	EC	zero_shot	0.12 / 0.54	0.14 / 0.48	0.10 / 0.40	2
		one_shot	0.00 / 0.00	0.06 / 0.01	0.00 / 0.00	0
		few_shot	0.03 / 0.23	0.01 / 0.01	0.00 / 0.02	0
	ECR	zero_shot♦	0.11 / 0.48	0.23 / 0.50	0.13 / 0.44	28
		one_shot♦	0.00 / 0.00	0.07 / 0.02	0.00 / 0.00	33
		few_shot	0.00 / 0.00	0.07 / 0.01	0.00 / 0.00	2
	EHC	zero_shot♦	0.17 / 0.67	0.15 / 0.54	0.11 / 0.41	97
		one_shot	0.07 / 0.47	0.08 / 0.03	0.01 / 0.03	5
		few_shot	0.10 / 0.57	0.10 / 0.02	0.03 / 0.03	3
gemma:7b	Direct	zero_shot	0.07 / 0.39	0.04 / 0.02	0.01 / 0.02	0
		one_shot♦	0.00 / 0.00	0.02 / 0.01	0.00 / 0.00	25
		few_shot♦	0.04 / 0.27	0.06 / 0.46	0.05 / 0.34	34
	EC	zero_shot	0.16 / 0.32	0.14 / 0.44	0.15 / 0.37	3
		one_shot♦	0.00 / 0.00	0.09 / 0.03	0.01 / 0.00	131
		few_shot♦	0.03 / 0.15	0.07 / 0.38	0.04 / 0.21	76
	ECR	zero_shot♦	0.14 / 0.36	0.22 / 0.46	0.14 / 0.40	47
		one_shot♦	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	208
		few_shot♦	0.04 / 0.14	0.09 / 0.38	0.05 / 0.21	143
	EHC	zero_shot	0.14 / 0.66	0.15 / 0.47	0.10 / 0.37	18
		one_shot♦	0.00 / 0.00	0.08 / 0.04	0.01 / 0.00	134
		few_shot♦	0.02 / 0.13	0.07 / 0.03	0.00 / 0.01	196

Figure 1: Results from Fechner and Dorpinghaus's study on LLM text classification using zero-shot, one-shot, and few-shot prompting.

2.2.2 Prompting Strategy

Due to time and computational constraints, the direct prompting method was used instead of the more advanced EHC method for prompt engineering which would have yielded better results. The following prompting structure was employed:

```
You are an expert in text classification. You must
   strictly choose one category
from the list of categories provided below.
If none of the categories match perfectly, choose
   the category that is the closest
in meaning or concept.
Do not invent new categories that are not listed.
   Only respond with a single
category from the provided list.
Sentence: "{sentence}"
NER Tags: {ner_dict}
Categories: {', '.join(self.categories)}
```

This prompt was designed to guide the LLM in selecting one of the 49 valid categories for each sentence. If the LLM did not return a valid category or failed to classify the sentence after three attempts, the sentence was cate-

gorized as "unknown." In total, 169 sentences were classified as "unknown" out of 16,000 test sentences.

2.3 Handling Class Imbalance

Due to the imbalanced nature of the dataset, **weighted performance metrics** such as **precision**, **recall**, and **F1 score** were used to evaluate the model's performance. These metrics provide a more accurate assessment of model performance in imbalanced datasets by accounting for the number of samples in each class.

- **Precision**: Measures the ratio of correctly predicted positive observations (true positives) to the total number of observations predicted as positive (true positives + false positives). It answers the question: "Of all the instances classified as a certain class, how many are actually of that class?"
- **Recall**: Measures the ratio of correctly predicted positive observations (true positives) to all observations that actually belong to the positive class (true positives + false negatives). It answers the question: "Of all the instances that truly belong to a certain class, how many were correctly classified?"
- **F1 Score**: The harmonic mean of precision and recall. It provides a balance between precision and recall, especially useful when the dataset is imbalanced. The formula is:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

2.4 Results

2.4.1 Overall Results

The following table presents the overall performance metrics for the test set using the Llama 3:8B model in zero-shot mode with direct prompting:

These results reflect moderate performance, with test accuracy at around 42.41%, while the balanced accuracy of 26.66% indicates challenges in correctly classifying the less represented classes due to dataset imbalance.

Next, we will compare these results with Fechner and Dorpinghaus's findings.

Metric	Value
Test Accuracy	42.41%
Balanced Accuracy	26.66%
Test Precision	43.82%
Test Recall	42.41%
Test F1 Score	40.83%

Table 1: Overall performance metrics for Llama 3:8B with zero-shot and direct prompting

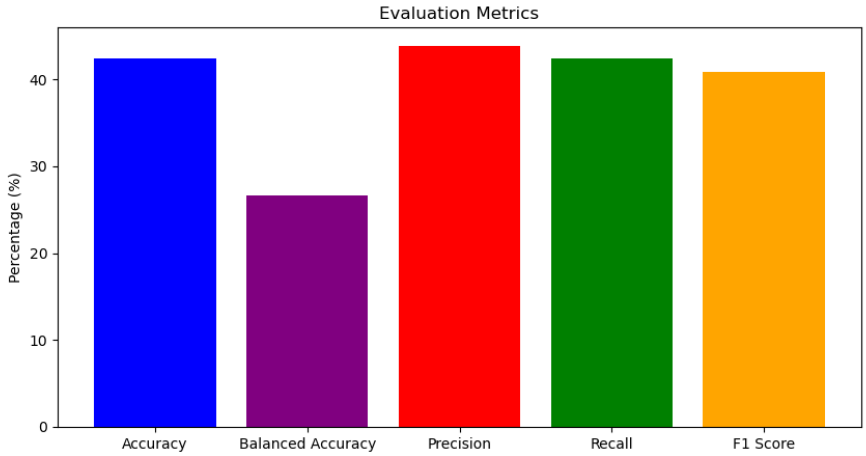

Figure 2: Overall performance metrics

2.4.2 Comparison with Fechner's Study

Fechner and Dorpinghaus's study applied a similar approach using the Llama 3:8B model with zero-shot, one-shot, and few-shot prompting strategies. When comparing my results to theirs, we observe similar trends, where the zero-shot method struggles with class imbalances, achieving lower precision and recall metrics. In particular, their study shows that the zero-shot method does not perform as well for imbalanced datasets, which is consistent with my findings. However, it is important to note that my results demonstrate slightly better overall performance, particularly in terms of precision and F1 score.

Additionally, they did not encounter any failed samples during their evaluation in the zero-shot method, which aligns with my findings, as I also did not experience any failed samples in this mode. Despite the challenges posed

by the imbalanced dataset, my results show improvements over their study in some aspects, particularly in handling categories with fewer samples.

LLM	Strategy	Prompt Type	Metrics (Macro/Weighted)			Failed Samples
			Precision	Recall	F1 Score	
	Direct	zero_shot	0.01 / 0.00	0.12 / 0.01	0.02 / 0.00	0
		one_shot	0.04 / 0.13	0.03 / 0.01	0.01 / 0.02	0
		few_shot	0.11 / 0.61	0.05 / 0.16	0.04 / 0.19	2

Figure 3: Comparison of results from Fechner and Dorpinghaus's study on Llama 3:8B using zero-shot prompting.

2.4.3 Class-wise Results

The following table shows the class-wise performance metrics, including precision, recall, F1-score, and support (number of samples per class) for each class. These results are from the test set using Llama 3:8B with zero-shot prompting.

Class	**Precision**	**Recall**	**F1-Score**	**Support**
music	0.31	0.16	0.21	117
royalty	0.58	0.92	0.71	130
religion	0.63	0.38	0.47	50
location	0.52	0.53	0.52	95
tv	0.43	0.14	0.21	385
people	1.00	0.01	0.02	80
biology	0.42	0.84	0.56	518
sports	0.31	0.23	0.27	548
organization	0.11	0.07	0.08	213
medicine	0.40	0.09	0.15	134
...	...	...	...	...
unknown	0.24	0.18	0.21	88
accuracy	-	-	0.42	16,371
macro avg	0.31	0.26	0.24	16,371
weighted avg	0.44	0.42	0.41	16,371

Table 2: Class-wise performance metrics for Llama 3:8B with zero-shot prompting

The class-wise results show a significant imbalance in the model's performance across different categories. For example, the model performs well in the "royalty" and "biology" categories, achieving an F1-score of 0.71 and 0.56, respectively. However, it struggles with "people" (F1-score of 0.02), "sports" (F1-score of 0.27), and other smaller categories, reflecting the challenge of classifying under-represented classes in the dataset.

2.4.4 Confusion Matrix

The confusion matrix provides a visual representation of how well the model predicts each class. The rows represent the actual labels, while the columns

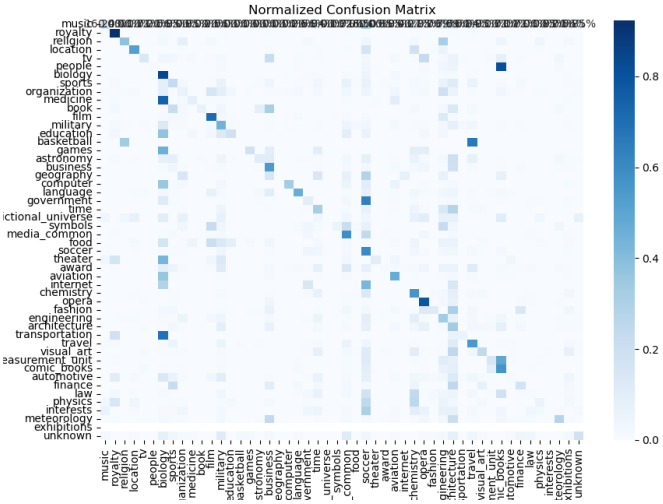

Figure 4: Confusion Matrix for Llama 3:8B with zero-shot prompting

represent the predicted labels. Here is a small portion of the confusion matrix for better understanding:

$$
\begin{matrix}
19 & 0 & 0 & \dots & 0 & 3 & 1 \\
1 & 120 & 0 & \dots & 0 & 0 & 0 \\
0 & 0 & 19 & \dots & 0 & 1 & 0 \\
\vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\
0 & 5 & 0 & \dots & 149 & 2 & 0 \\
0 & 0 & 0 & \dots & 0 & 0 & 0 \\
2 & 0 & 0 & \dots & 0 & 2 & 16
\end{matrix}
$$

The confusion matrix reveals that the model tends to predict a few dominant categories more accurately while frequently misclassifying instances from under-represented classes. This further highlights the model's limitations when applied to imbalanced datasets. For example, the category "royalty" shows strong diagonal values (high correct predictions), while "people" and "fictional universe" show poor predictions, as indicated by low values along the diagonal.

The combination of weighted metrics, class-wise performance, and the confusion matrix shows that while the model can perform well in major classes, it struggles with under-represented ones, resulting in lower balanced accuracy and F1-scores for these categories.

2.5 Discussion

The test results indicate that the Llama 3:8B model performed moderately well in some categories but poorly in others. The overall test accuracy was **42.41%**, with a balanced accuracy of **26.66%**, reflecting the difficulties of classifying imbalanced data. Categories such as **sports**, **architecture**, and **biology** achieved better precision and recall, while other categories like **fictional universe**, **media common**, and **people** struggled. The direct prompting strategy without fine-tuning faced limitations in this scenario.

Comparing these results to the study by Fechner and Dorpinghaus, it becomes evident that direct prompting with Llama 3:8B is suboptimal for highly imbalanced datasets like this one. They found similar issues in their zero-shot classifications, though the weighted metrics improved slightly when utilizing one-shot and few-shot techniques.

In the next phase, we will explore traditional ML/DL techniques and compare their performance against these LLM results.

3 Machine Learning: Logistic Regression

3.1 Data Analysis and Preprocessing

In this section, we followed similar steps for data analysis and preprocessing as described in the LLM section. However, for this part of the study, I used the cleaned sentences for training the logistic regression model. Since the task at hand is category classification, punctuation was not considered to contribute to the model's performance, and therefore it was removed during preprocessing.

The data was split into a 90% training and 10% test set and saved in TSV files named *cleaned_train_data_ml.tsv* and *cleaned_test_data_ml.tsv*. These preprocessing steps are implemented in the *dataset_ml.py* file.

3.2 Model Choice and Training

I chose Logistic Regression as the ML technique for this part of the study. According to research conducted by Suneera C M and Jay Prakash in their study titled *Performance Analysis of Machine Learning and Deep Learning Models for Text Classification*, Logistic Regression exhibited the highest accuracy among all the ML techniques they compared. Here, I will include two images from their study that show the accuracy of various ML and DL techniques on text classification tasks. Based on their results, Logistic Regression was selected for its relatively high accuracy in similar tasks.

TABLE I
CLASSIFICATION WITH TF-IDF DOCUMENT REPRESENTATION

ML Method	Accuracy	
	tf-idf_reduced	*tf-idf_complete*
Logistic Regression	78.98	**82.74**
Support Vector Machines	78.65	82.53
Random Forest	73.04	76.13
KNN	61.11	65.92
Naive Bayes	58.44	77.39
Decision Tree	54.95	55.60

TABLE IV
CLASSIFICATION WITH DEEP LEARNING METHODS

DL Model	*Accuracy*
Multi Layer Perceptron	78.74
Convolutional Neural Network	**80.27**
Long Short Term Memory Network	70.41
Bidirectional LSTM	72.11
CNN+LSTM	72.69

Figure 5: Results from Prakash and Suneera's study on ML and DL techniques for text classification

3.3 Preprocessing for Model Training

Before training the Logistic Regression model, I applied additional preprocessing steps:

- **Stopword Removal**: Since this is a category classification task rather than sentiment analysis, I removed stopwords from the cleaned sentences in both the training and test data. I used a custom stopword list that I obtained from a study, which included common Turkish stopwords like *'fakat', 'lakin', 'ancak', 'acaba'* and others. These stopwords do not contribute meaningfully to category classification.

- **TF-IDF Vectorization**: I applied Term Frequency-Inverse Document Frequency (TF-IDF) vectorization to transform the text data into numerical features. TF-IDF captures the importance of words in the context of the entire corpus. The formula for TF-IDF is:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

  Where TF is the term frequency of term t in document d, N is the total number of documents, and DF is the document frequency of term

t. This vectorization process converts the text into a format suitable for model training.

- **Label Encoding**: I encoded the categorical labels into numerical values to facilitate model training. This step is necessary because ML algorithms require numerical input for classification tasks.

3.4 Training the Logistic Regression Model

Logistic Regression was trained on the TF-IDF vectorized data, using the following parameters:

- **max_iter=1000**: I set the maximum number of iterations to 1000. This choice was made based on the study by Suneera and Prakash, where they used similar settings in their experiments. Increasing the maximum iterations ensures convergence in complex tasks like multi-class classification.

- **class_weight='balanced'**: Since the dataset is imbalanced, I used this parameter to automatically adjust weights inversely proportional to class frequencies. This ensures that the model pays more attention to minority classes, improving overall performance.

- **multi_class='multinomial'**: I used this option for multinomial logistic regression, which is more suitable for multi-class classification tasks like ours.

During training, I observed the following details, which were reported in the model output:

```
RUNNING THE L-BFGS-B CODE

Machine precision = 2.220D-16
 N =  25429187     M =  10
 This problem is unconstrained.

At X0         0 variables are exactly at the bounds

At iterate    0    f=  3.89182D+00    |proj g|=  1.98779D-03
At iterate   50    f=  1.57374D+00    |proj g|=  4.57283D-03
At iterate  100    f=  1.42322D+00    |proj g|=  1.73798D-03
At iterate  150    f=  1.38872D+00    |proj g|=  7.58141D-04
```

```
N    Tit     Tnf  Tnint  Skip  Nact     Projg        F
*****    184    198      1     0     0   9.829D-05   1.385D+00
F = 1.3847788134447274

CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL
```

The L-BFGS-B optimization algorithm was used to minimize the loss function during logistic regression training. It began with a high initial loss value of 3.89 and progressively reduced the loss to 1.38 over 184 iterations. The process utilized 198 function evaluations and successfully converged based on the projected gradient norm, which became smaller than a set tolerance (9.829e-05). The convergence message indicates that further improvements to the loss function were minimal, and the algorithm found a near-optimal solution. The optimization was unconstrained, meaning no bounds were applied to the parameters. Overall, the training was completed without issues, achieving successful convergence and minimizing the model's loss.

Unfortunately, due to computational constraints, I was unable to experiment with different hyperparameter values or perform cross-validation with a 10% validation split as initially planned. However, class imbalance was handled using the 'class_weight='balanced'' hyperparameter during training.

3.5 Results

The following are the overall performance metrics for the Logistic Regression model. These metrics are particularly suited for imbalanced datasets:

Metric	Value
Test Precision	62.28%
Test Recall	53.56%
Test F1-Score	53.87%
Test Accuracy	53.56%
Balanced Accuracy	63.89%

Table 3: Overall performance metrics for Logistic Regression

These results show an improvement over the Llama 3:8B model in terms of overall accuracy and F1-Score, particularly in handling the imbalanced dataset. However, they also reflect the challenges of correctly classifying underrepresented categories.

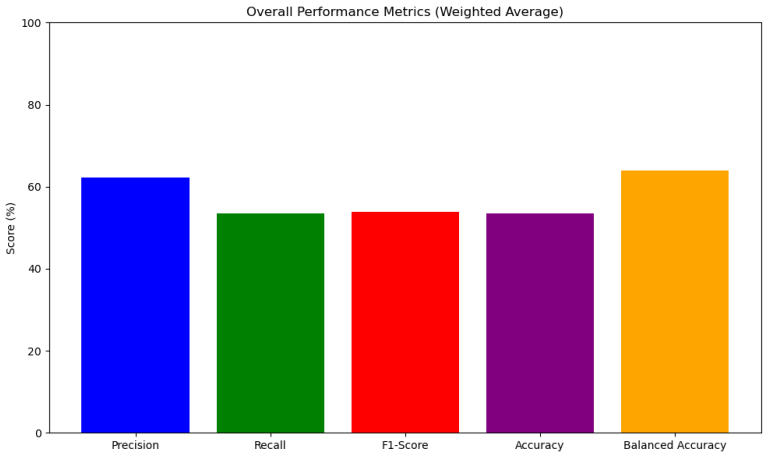

Figure 6: Overall performance metrics for Logistic Regression

3.6 Class-wise Report

The class-wise report for the Logistic Regression model is shown below, including precision, recall, F1-score, and support for each class:

Class	Precision	Recall	F1-Score	Support
0	0.28	0.56	0.37	502
1	0.81	0.91	0.86	483
2	0.59	0.89	0.71	231
3	0.61	0.81	0.70	399
4	0.38	0.45	0.41	1426
5	0.36	0.79	0.49	388
6	0.75	0.78	0.77	1939
...	...	...	...	...
27	0.85	0.48	0.61	9141
28	0.52	0.76	0.62	123
29	0.20	0.43	0.27	654
...	...	...	...	...
45	0.56	0.74	0.64	453
46	0.11	0.35	0.16	341
47	0.58	0.62	0.60	2086
48	0.32	0.63	0.43	405
accuracy			0.54	
macro avg	0.47	0.64	0.53	65484
weighted avg	0.62	0.54	0.54	65484

Table 4: Class-wise report for Logistic Regression (Selected Classes)

3.7 Confusion Matrix

The confusion matrix shows the distribution of correct and incorrect classifications made by the Logistic Regression model. Here is a summary of the confusion matrix for better understanding:

282	1	0	1	3	1	4
0	438	0	0	1	0	3
1	0	205	2	0	0	0
...	...	...	...	...	...	...

In the confusion matrix, the rows represent the actual labels, and the columns represent the predicted labels. Most of the correct predictions are located along the diagonal, with some off-diagonal entries indicating misclassifications.

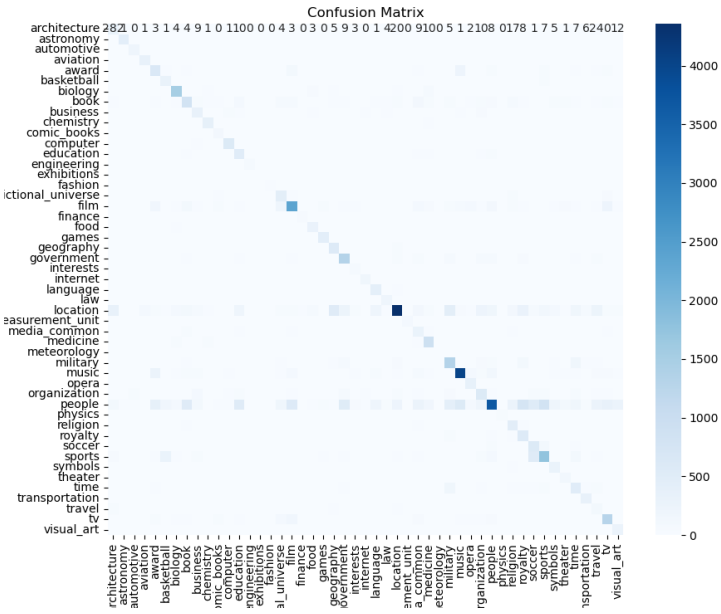

Figure 7: Confusion Matrix for Logistic Regression

The confusion matrix reveals that Logistic Regression performs well for the majority classes but struggles with the minority classes, as indicated by lower recall and precision in the class-wise report.

3.8 Comparison with Other Studies

The results obtained in the ML technique section were compared to the study by Suneera C M and Jay Prakash, *Performance Analysis of Machine Learning and Deep Learning Models for Text Classification.* In their study, Logistic Regression achieved an accuracy of 82.74% (as shown in Figure 5), which is

higher than the balanced accuracy of 63.89% achieved in my study (see Figure 6). This difference can be attributed to the specific characteristics of the dataset used in each study. My dataset is more imbalanced, which posed additional challenges for classification performance. Nonetheless, Logistic Regression in both cases confirmed its strength in handling multi-class classification tasks effectively, particularly when class imbalance is addressed, as in my case using the `class_weight='balanced'` parameter.

4 Discussion and Concusion : LLM vs. ML Method for Text Classification

In this part, I compared the results of the Large Language Model (LLM) approach, specifically using the Llama 3:8B model, and the Machine Learning (ML) approach using Logistic Regression. Both methods were applied to the same dataset with a focus on handling imbalanced classes. Below, I summarized and discuss the strengths, weaknesses, and performance of each method in this text classification task.

4.1 Performance Comparison

The overall accuracy and other performance metrics from both the LLM and ML approaches show notable differences:

Metric	**Llama 3:8B (LLM)**	**Logistic Regression (ML)**
Test Accuracy	42.41%	53.56%
Balanced Accuracy	26.66%	63.89%
Precision	43.82%	62.28%
Recall	42.41%	53.56%
F1 Score	40.83%	53.87%

Table 5: Overall performance metrics for LLM and ML methods

From Table 5, it is evident that the Logistic Regression model (ML method) outperforms the Llama 3:8B (LLM method) in terms of all key metrics, especially in handling the imbalanced dataset, as reflected by the higher balanced accuracy and F1-score. The ML approach's ability to better handle class imbalance can be attributed to the use of the 'class_weight='balanced'' parameter, which gave more weight to the underrepresented classes during training. However, it is also important to note that the performance of the

LLM method could have been affected by the unfortunate accident I experienced, which led to computational constraints and forced me to use only 2.5% of the data for testing in the LLM model, potentially limiting its performance compared to the ML method.

4.2 Advantages and Limitations of LLM Approach

The Llama 3:8B model, using a zero-shot direct prompting strategy, exhibited moderate performance, with an overall test accuracy of 42.41%. Although it performed reasonably well for some larger categories like *biology* and *royalty*, the model struggled significantly with underrepresented classes. This is particularly reflected in the low balanced accuracy of 26.66%, which highlights its difficulty in correctly classifying minority classes.

One major strength of the LLM approach is that it does not require task-specific training, which makes it attractive for scenarios where computational resources are limited, and fine-tuning is not feasible. However, its performance on imbalanced datasets suffers without further prompt engineering like EHC prompt strategy or few-shot prompt type learning, which could mitigate these issues. Additionally, the zero-shot approach may not be ideal for highly specialized tasks like this category-based text classification, where the model needs to differentiate between many fine-grained classes.

4.3 Advantages and Limitations of ML Approach

In contrast, Logistic Regression, a traditional ML method, showed better performance, particularly for imbalanced data. With a balanced accuracy of 63.89% and a higher F1-score, the ML method was able to correctly classify more of the underrepresented classes than the LLM approach. The ‘class_weight='balanced'‘ parameter allowed the model to give more importance to less frequent categories, improving the overall classification performance.

However, the ML approach required more preprocessing, including stopword removal, TF-IDF vectorization, and label encoding. Furthermore, Logistic Regression is inherently more reliant on feature engineering. Despite these limitations, Logistic Regression demonstrated superior accuracy and better handling of class imbalance in this specific case study.

4.4 Computational Efficiency

Contrary to initial expectations, the LLM method using the Llama 3:8B model required significantly more time to process the data compared to the

Logistic Regression model even though the test data was far more small when using the LLM method. Although the Llama 3:8B model was accessed via an API and relied on pre-trained weights, the process of generating predictions with large-scale LLMs was computationally intensive and time-consuming. This was especially true for handling the large test set and multiple prompts per sentence that is why I had to shrink the size of the test dataset and could not use EHC prompt strategy. In contrast, the Logistic Regression model, while requiring data preprocessing and several iterations of optimization during training, was still faster overall in generating predictions once trained.

Therefore, in scenarios where computational resources and time are limited, the traditional ML approach (Logistic Regression) may be more efficient, especially when dealing with large datasets. Although the LLM approach benefits from not requiring task-specific training, its computational overhead can outweigh its advantages in such cases, particularly for imbalanced and large datasets.

4.5 Practicality and Use Case Discussion

The practicality of each method depends largely on the deployment context. In a scenario where pre-trained language models can be leveraged with minimal customization, the LLM approach might be a better fit, especially if the classification task is less granular or if the dataset is not highly imbalanced. However, for more fine-grained classification tasks, like the one explored in this study, the traditional ML approach, with additional handling for imbalanced datasets, offers better performance.

Given these findings, Logistic Regression would be the recommended method for deployment in this case study, given its superior handling of the dataset's class imbalance and its relatively high accuracy and F1-scores.

4.6 Conclusion

In conclusion, both the LLM and ML methods have their merits and limitations. The Llama 3:8B model, while offering the ability to classify text without task-specific training, struggled with both the imbalanced nature of the dataset and the computational demands of processing large-scale language models. Despite using an API and pre-trained weights, the LLM approach required significantly more time to generate predictions compared to the Logistic Regression model. Logistic Regression, on the other hand, provided better overall performance by directly addressing class imbalance

through balanced weighting, and it was also computationally more efficient for this task.

The choice between these methods should therefore be driven by the specific requirements of the task. If higher performance, particularly for imbalanced datasets, and computational efficiency are priorities, traditional ML methods like Logistic Regression may be preferable. However, in cases where leveraging pre-trained language models without further fine-tuning is necessary, and the computational resources are sufficient, LLMs can offer a viable alternative, though at a higher computational cost.

5 References

1. Fechner and Dorpinghaus, *No Train, No Pain? Assessing the Ability of LLMs for Text Classification with no Finetuning.* https://annals-csis.org/proceedings/2024/pliks/9098.pdf

2. C. M. Suneera and J. Prakash, "Performance Analysis of Machine Learning and Deep Learning Models for Text Classification," 2020 IEEE 17th India Council International Conference (INDICON), New Delhi, India, 2020, pp. 1-6, doi: 10.1109/INDICON49873.2020.9342208.